

POLITECHNIKA WROCŁAWSKA	
Wydział Informatyki i Telekomunikacji	
Algorytmy i złożoność obliczeniowa	
Rok akademicki:	2024/2025
Autor projektu:	Kacper Piekut
Nr indeksu:	280879
Temat projektu:	Zadanie 1: Badanie efektywności algorytmów sortowania

Oświadczenie: Przekazując to sprawozdanie do oceny prowadzącemu zajęcia Autor oświadcza, że zostało ono przygotowane samodzielnie, bez udziału osób trzecich oraz że żadna jego część nie jest plagiatem.

1. Wstęp

Sortowanie jest jednym z fundamentalnych problemów algorytmicznych w informatyce, a jego efektywne rozwiązanie ma kluczowe znaczenie w wielu dziedzinach, takich jak analiza danych, przetwarzanie informacji, optymalizacja czy systemy bazodanowe. Wybór odpowiedniego algorytmu sortowania zależy od wielu czynników, takich jak rozmiar danych, ich rozkład, dostępne zasoby sprzętowe oraz wymagania dotyczące czasu i pamięci. Algorytmy sortowania można podzielić na dwie główne kategorie: sortowania porównawcze i sortowania nie porównawcze. W przypadku sortowań porównawczych, takich jak sortowanie przez wstawianie, elementy są porównywane parami w celu ustalenia ich właściwej kolejności. Dolna granica złożoności czasowej dla algorytmów porównawczych wynosi $O(n \log n)$. Takie sortowania jak przez wstawianie, są często bardziej efektywne dla małych zbiorów danych lub danych częściowo posortowanych. Działają one na zasadzie iteracyjnego wstawiania elementów do odpowiedniego miejsca w już posortowanej części tablicy. W najgorszym przypadku, gdy dane są posortowane w odwrotnej kolejności, algorytm musi przesunąć każdy element, wtedy złożoność wynosi $O(n^2)$. Wariant sortowania przez wstawianie binarne optymalizuje proces wyszukiwania miejsca wstawienia, wykorzystując algorytm wyszukiwania binarnego. Sortowanie przez wstawianie jest stabilnym algorytmem, co oznacza, że nie zmienia kolejności elementów o równych kluczach. Jest to istotne w przypadku sortowania danych, w których zachowanie pierwotnej kolejności ma znaczenie, np. w sortowaniu rekordów według jednego z pól. Algorytm ten jest również prosty w implementacji i ma niewielkie wymagania pamięciowe, co czyni go popularnym wyborem w przypadku małych zbiorów danych lub jako część bardziej złożonych algorytmów. W ramach tego projektu zaimplementowano i przeanalizowano efektywność następujących algorytmów sortowania:

- **Sortowanie przez wstawienie liniowe** - w czterech wariantach (oznaczonych jako **sortA**, **sortB**, **sortC**, **sortD**).
- **Sortowanie przez wstawienie binarne** - z wykorzystaniem binarnego wyszukania miejsca wstawienia (**sortE**).

Algorytmy te należą do grupy sortowań opartych na porównaniach, co oznacza, że ich działanie polega na porównywaniu elementów w celu ustalenia ich właściwej kolejności.

Teoretyczne złożoności czasowe zaimplementowanych algorytmów są następujące:

- **Sortowanie przez wstawienie liniowe:**
 - W przypadku danych losowych średnia złożoność powinna wynosić **$O(n^2)$** .
 - W przypadku danych już posortowanych najlepszy przypadek powinien wynosić **$O(n)$** .
 - W przypadku danych posortowanych w odwrotnej kolejności najgorszy przypadek powinien wynosić **$O(n^2)$** .
- **Sortowanie przez wstawienie binarne:**
 - Wyszukiwanie miejsca wstawienia ma złożoność **$O(\log n)$** , ale przesuwanie elementów nadal wymaga **$O(n)$** , co daje łączną złożoność **$O(n^2)$** .

Celem projektu jest nie tylko implementacja tych algorytmów, ale także ich analiza pod kątem efektywności w różnych warunkach. W szczególności badane są:

- czas działania algorytmów w zależności od rozmiaru danych,
- liczba porównań i przestawień elementów w tablicy,
- wpływ rozkładu danych wejściowych (losowe, posortowane rosnąco, posortowane malejąco) na wydajność.

Wszystkie algorytmy zostały zaimplementowane od podstaw, bez użycia gotowych bibliotek, zgodnie z wymaganiami zadania. W kolejnych sekcjach przedstawiono cele projektu, szczegóły eksperymentu, wyniki pomiarów oraz wnioski dotyczące efektywności poszczególnych metod.

2. Cele projektu

- Porównanie efektywności algorytmów sortowania - zbadanie, jak różne implementacje sortowania przez wstawianie (liniowe i binarne) radzą sobie z różnymi typami danych wejściowych (losowe, posortowane rosnąco, posortowane malejąco).
- Zrozumienie wpływu rozkładu danych na wydajność algorytmów - sprawdzenie, jak różne rozkłady danych (np. losowe, uporządkowane) wpływają na czas działania i liczbę operacji.
- Weryfikacja teoretycznych założeń złożoności czasowych - porównanie wyników eksperymentalnych z teoretycznymi złożonościami czasowymi algorytmów.
- Tworzenie narzędzi do analizy danych - opracowanie programu, który umożliwia automatyczne testowanie i generowanie wyników.

3. Plan eksperymentu

Algorytmy do testowania

W eksperymencie analizowane są następujące algorytmy sortowania:

- **sortA** - klasyczna implementacja sortowania przez wstawianie, w której elementy są przesuwane w lewo w celu znalezienia odpowiedniego miejsca dla wstawianego elementu.
- **sortB** - wariant sortowania przez wstawianie, w którym najpierw wyszukiwane jest miejsce wstawienia, a następnie elementy są przesuwane w prawo.
- **sortC** - odwrócona wersja klasycznego sortowania przez wstawianie, w której iteracja odbywa się od końca tablicy do początku.
- **sortD** - wariant sortowania przez wstawianie, w którym elementy są przesuwane w prawo, ale iteracja odbywa się od końca tablicy.
- **sortE** - implementacja sortowania przez wstawianie, w której miejsce wstawienia elementu jest wyszukiwane za pomocą algorytmu wyszukiwania binarnego, co zmniejsza liczbę porównań, ale nadal wymaga przesuwania elementów.

Dane wejściowe

Dane wejściowe do testowania algorytmów są generowane w trzech różnych rozkładach:

- Losowe dane - tablica jest wypełniona liczbami całkowitymi generowanymi za pomocą funkcji **rand**. Liczby są losowane z zakresu od 0 do 99 999. Ten typ danych symuluje przypadkowy rozkład elementów.
- Dane posortowane rosnąco - tablica jest wypełniona liczbami w kolejności rosnącej (od **0** do **n-1**). Ten typ danych reprezentuje najlepszy przypadek dla algorytmów sortowania przez wstawianie.
- Dane posortowane malejąco - tablica wypełniona liczbami w kolejności malejącej od (**n-1** do **0**). Ten typ danych reprezentuje najgorszy przypadek dla algorytmów sortowania przez wstawianie.

Rozmiary tablic

Eksperyment przeprowadzany jest dla tablic o różnych rozmiarach, aby zbadać wpływ wielkości danych na czas działania i liczbę operacji. Rozmiary tablic to: 1000, 5000, 10000, 50000 oraz 100000. Rozmiary te zostały dobrane tak, aby umożliwić analizę zarówno dla małych, jak i dużych zbiorów danych, przy jednoczesnym uwzględnieniu ograniczeń sprzętowych.

Pomiar wyników

- Czas działania - mierzony jest za pomocą funkcji **clock** z biblioteki **<time.h>**. Wynik jest podawany w milisekundach, co pozwala na precyzyjne porównanie wydajności algorytmów.
- Liczba porównań - zliczana jest w zmiennej **comps** w każdej implementacji algorytmu. Liczba porównań obrazuje, jak często algorytm sprawdza relacje między elementami.
- Liczba przestawień - zliczana jest w zmiennej **swaps** w każdej implementacji algorytmu. Liczba przestawień pokazuje, ile razy elementy tablicy zostały fizycznie przesunięte w pamięci.

Weryfikacja poprawności

Po każdym sortowaniu wynik jest sprawdzany za pomocą funkcji **isSorted**, która weryfikuje, czy tablica została poprawnie posortowana w porządku rosnącym. W przypadku błędu wyświetlany jest komunikat ostrzegawczy.

Środowisko testowe

Eksperyment jest wykonywany w środowisku konsolowym na komputerze z systemem Windows. Kod został napisany w języku C++. Wyniki mogą być zależne od wydajności sprzętu, dlatego wnioski dotyczące efektywności algorytmów są formułowane w sposób ogólny.

Specyfikacja sprzętowa

Laptop: ASUS TUF Gaming F15.

Procesor: Intel Core i5-11400H, 6 rdzeni, 12 wątków, taktowanie bazowe 2.7 GHz (do 4.5 GHz w trybie turbo).

Pamięć RAM: 16 GB DDR4.

Dysk: SSD NVMe 512 GB (Samsung MZVLQ512HBLU-00B00).

Karta graficzna: NVIDIA GeForce RTX 3050 Laptop GPU oraz Intel UHD Graphics.

System operacyjny: Microsoft Windows 11 Home 64-bit, wersja 10.0.26100.

Oczekiwane wyniki (hipotezy)

- **H1** - Algorytmy sortowania przez wstawianie liniowe (**sortA**, **sortB**, **sortC**, **sortD**) powinny wykazywać złożoność $O(n^2)$ w przypadku danych losowych i malejących, oraz $O(n)$ w przypadku danych rosnących.
- **H2** - Algorytm sortowania przez wstawianie binarne (**sortE**) powinien zmniejszyć liczbę porównań dzięki wyszukiwaniu binarnemu, ale liczba przestawień pozostanie taka sama jak w przypadku innych algorytmów, co nadal daje złożoność $O(n^2)$.
- **H3** - Przy danych wejściowych posortowanych rosnąco liczba przestawień powinna wynosić **0**, a liczba porównań **$n - 1$** (bez uwzględnienia sortowania binarnego).

4. Implementacja

Struktura programu

- Program zawiera pięć funkcji (**sortA**, **sortB**, **sortC**, **sortD**, **sortE**) implementujących różne warianty algorytmu sortowania przez wstawianie. Funkcje zostały przedstawione i opisane wyżej.
- Funkcja sprawdzająca poprawność - **isSorted**. Po każdym sortowaniu wynik jest weryfikowany za pomocą tej funkcji, która sprawdza, czy tablica jest posortowana w porządku rosnącym.
- Funkcja pomiaru wyników - **displayAndSort**. Funkcja ta odpowiada za wywoływanie algorytmów sortowania, pomiar czasu działania, zliczanie liczby porównań i przestawień oraz zapis pomiarów (wyników) do odpowiednich tablic.
- Główna funkcja programu - **main**, która zarządza całym procesem testowania algorytmów sortowania. Poniżej znajduje się opis jej zawartości:

- deklaracja parametrów testowych
- pętla determinująca rozmiar tablicy
- inicjalizacja tablic (**T** jest pierwotną i nieposortowaną tablicą)
- inicjalizacja pomiarów (wyników)
- pętla wybierająca typ danych wejściowych
- pętla generująca testy (generowanie danych wejściowych, wywołanie funkcji sortujących, uśrednienie pomiarów i wyświetlanie ich)
- usunięcie tablic

Uwagi

- Kod został odpowiednio skomentowany, aby ułatwić jego zrozumienie i analizę.
- Wszystkie algorytmy zostały zaimplementowane w sposób modułarny, co ułatwia ich testowanie i porównywanie.
- Program został zoptymalizowany pod kątem wydajności, ale zachowano prostotę implementacji, aby skupić się na analizie algorytmów.
- Część kodu w funkcji **displayAndSort** jest za komentowana - używana była w celu sprawdzenia sortowanej tablicy oraz poprawności zapisywanych pomiarów.
- Dla danych wejściowych posortowanych rosnąco i malejąco został wykonany tylko jeden test, ponieważ wyniki w kolejnych byłyby identyczne.
- Tabele wyświetlające pomiary nie są równe, ponieważ dane są pooddzielane tabulatorami dla wygodnego przenoszenia do programu Excel.

5. Zestawienie wyników

W ramach przeprowadzonych badań dokonano pomiarów efektywności sześciu algorytmów sortowania, w tym czterech wariantów sortowania przez wstawianie liniowe oraz sortowania przez wstawianie binarne. Pomiary dokonano dla różnych rozmiarów tablic (od 1000 do 100 000 elementów), w zależności od możliwości sprzętowych. Dla każdego przypadku wygenerowano dane wejściowe w trzech wariantach: losowe, posortowane rosnąco oraz malejąco. Dla zapewnienia wiarygodności pomiarów, każdy test był wykonywany 10-krotnie na nowo wygenerowanych danych i wyniki zostały uśrednione. Mierzono nie tylko czasu (w **ms**), ale również liczbę porównań oraz liczbę przestawień elementów, co pozwala na pełniejszą ocenę złożoności praktycznej algorytmów.

Wyniki dla danych wejściowych losowych

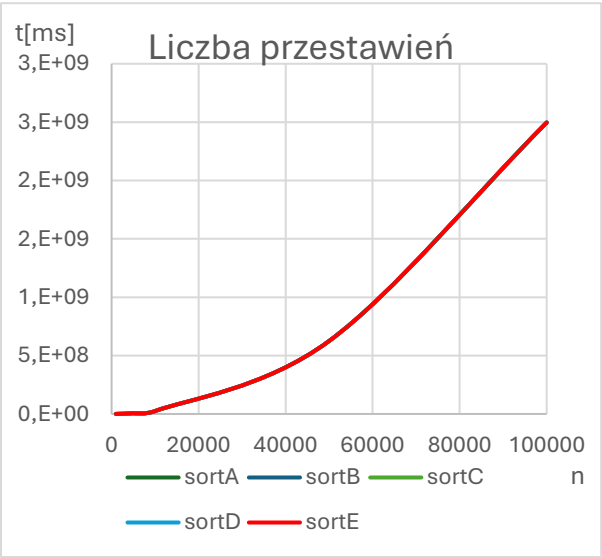
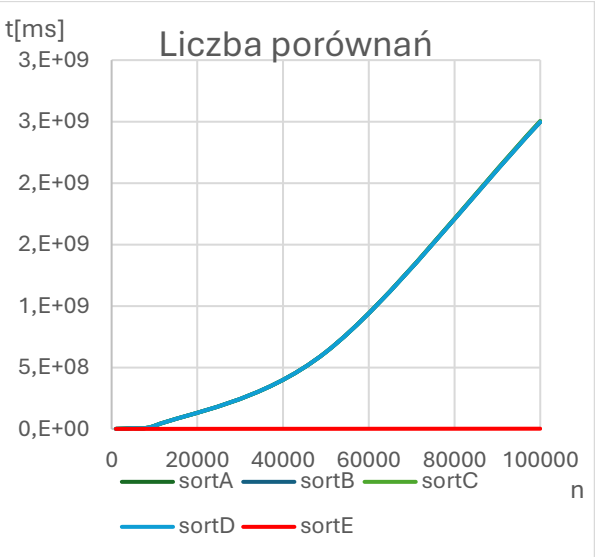
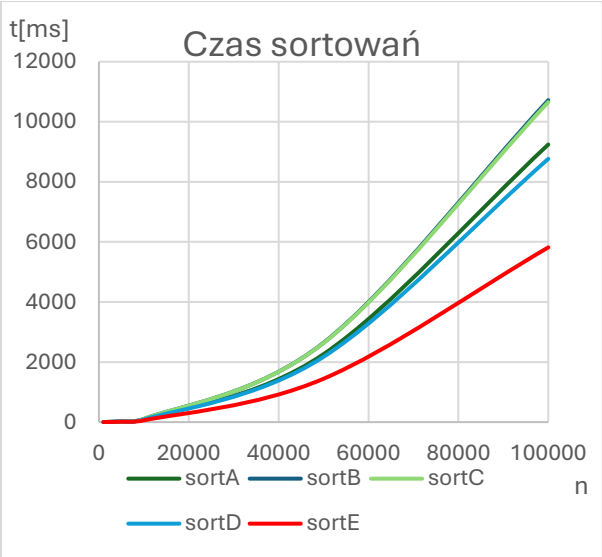
sortA			
n	t[ms]	comps	swaps
1000	1,0	252572	252573
5000	22,6	6247626	6247626
10000	92,0	25024471	25024473
50000	2263,4	624975599	624975599
100000	9242,2	2496378933	2496378930

sortB			
n	t[ms]	comps	swaps
1000	1,4	248913	252573
5000	26,8	6259856	6247626
10000	105,6	24990509	25024473
50000	2642,9	625099377	624975599
100000	10719,5	2503771040	2496378930

sortC			
n	t[ms]	comps	swaps
1000	1,2	249813	252674
5000	26,7	6264355	6248125
10000	105,6	24999508	25025472
50000	2634,0	625144378	624980599
100000	10655,2	2503861041	2496388931

sortD			
n	t[ms]	comps	swaps
1000	0,9	252572	252573
5000	22,0	6247626	6247625
10000	86,3	25024471	25024472
50000	2174,9	624975598	624975598
100000	8765,6	2496378933	2496378931

sortE			
n	t[ms]	comps	swaps
1000	0,6	8587	252573
5000	14,4	54504	6247626
10000	59,1	119006	25024473
50000	1444,0	711416	624975599
100000	5816,9	1522728	2496378930



Wyniki dla danych posortowanych rosnąco

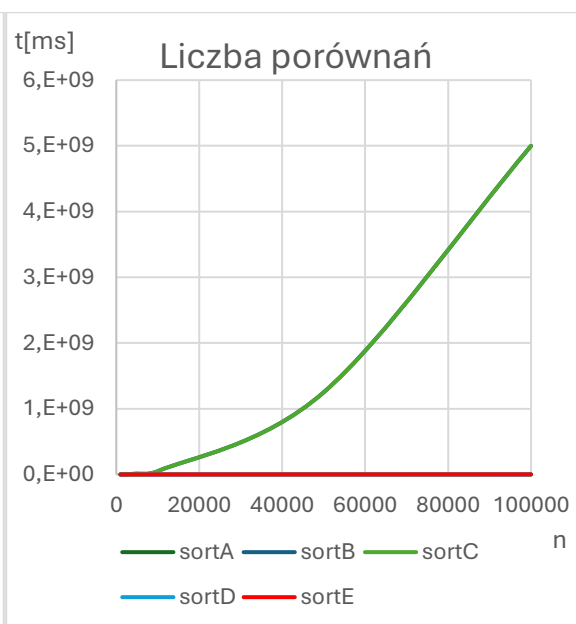
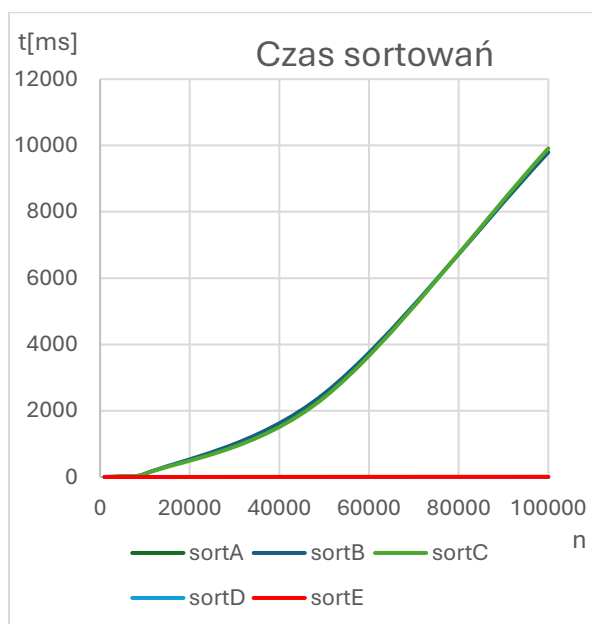
sortA			
n	t[ms]	comps	swaps
1000	0,0	999	0
5000	0,0	4999	0
10000	0,0	9999	0
50000	1,0	49999	0
100000	0,0	99999	0

sortB			
n	t[ms]	comps	swaps
1000	1,0	499500	0
5000	23,0	12497500	0
10000	94,0	49995000	0
50000	2513,0	1249975000	0
100000	9794,0	4999950000	0

sortC			
n	t[ms]	comps	swaps
1000	1,0	500500	0
5000	24,0	12502500	0
10000	95,0	50005000	0
50000	2398,0	1250025000	0
100000	9912,0	5000050000	0

sortD			
n	t[ms]	comps	swaps
1000	0,0	999	0
5000	0,0	4999	0
10000	0,0	9999	0
50000	0,0	49999	0
100000	0,0	99999	0

sortE			
n	t[ms]	comps	swaps
1000	0,0	8977	0
5000	0,0	56809	0
10000	0,0	123617	0
50000	3,0	734465	0
100000	6,0	1568929	0





Wyniki dla danych posortowanych malejąco

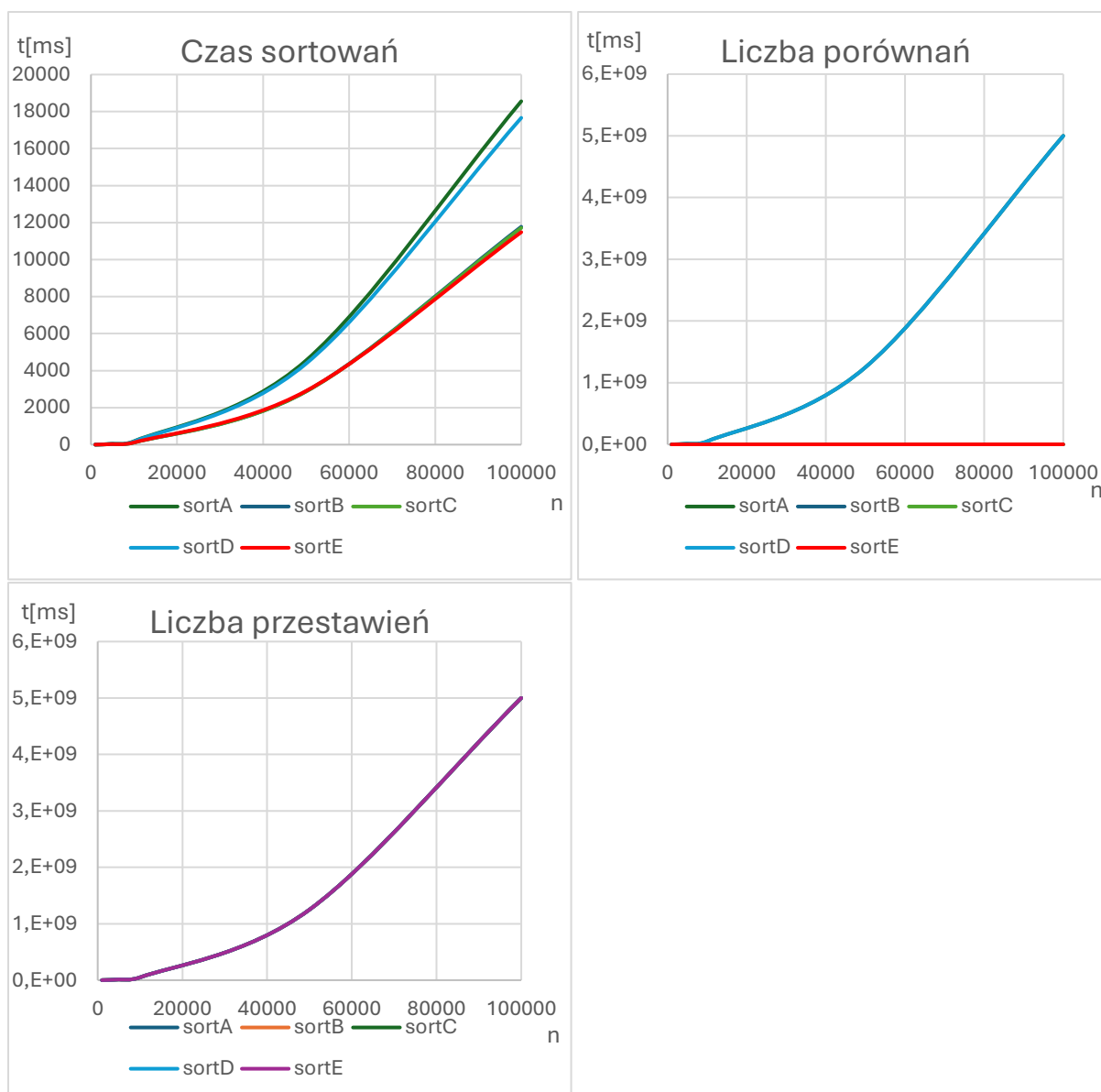
sortA			
n	t[ms]	comps	swaps
1000	2,0	499500	500499
5000	46,0	12497500	12502499
10000	183,0	49995000	50004999
50000	4551,0	1249975000	1250024999
100000	18557,0	4999950000	5000049999

sortB			
n	t[ms]	comps	swaps
1000	1,0	999	500499
5000	29,0	4999	12502499
10000	116,0	9999	50004999
50000	2877,0	49999	1250024999
100000	11775,0	99999	5000049999

sortC			
n	t[ms]	comps	swaps
1000	1,0	1999	500499
5000	29,0	9999	12502499
10000	115,0	19999	50004999
50000	2871,0	99998	1250024999
100000	11712,0	199998	5000049999

sortD			
n	t[ms]	comps	swaps
1000	2,0	499500	500499
5000	42,0	12497500	12502499
10000	171,0	49995000	50004999
50000	4376,0	1249975000	1250024999
100000	17660,0	4999950000	5000049999

sortE			
n	t[ms]	comps	swaps
1000	1,0	7987	500499
5000	29,0	51822	12502499
10000	117,0	113631	50004999
50000	2903,0	684481	1250024999
100000	11482,0	1468946	5000049999



Porównanie wyników

Na podstawie pomiarów z trzech zestawów danych można zauważyć różnice w efektywności algorytmów w zależności od rozkładu danych wejściowych. Sortowanie przez wstawianie binarne wykazuje wyraźnie mniejszą liczbę porównań, szczególnie w przypadku danych losowych, natomiast czas wykonania nie zawsze jest krótszy - co może być efektem dodatkowych operacji związanych z wyszukiwaniem pozycji wstawiania.

- Dane losowe - Algorytmy wykazały różną wydajność w zależności od implementacji, najlepszy czas wykonania osiągnął algorytm **sortE**, który wykorzystuje wyszukiwanie binarne do optymalizacji procesu wstawiania. Liczba porównań i przestawień była zgodna z teoretycznymi założeniami dla każdego algorytmu.
- Dane posortowane rosnąco - pomiary **sortA** i **sortD** są zgodne z teoretycznymi założeniami, liczba przestawień we wszystkich sortowaniach wyniosła **0**, co jest poprawnym wynikiem, bo „De facto” tablica już jest posortowana na wejściu.

- Dane posortowane malejąco - Dla tego przypadku wszystkie algorytmy wykazały największą liczbę porównań i przestawień, co jest zgodne z teoretycznymi założeniami. Algorytm **sortE** ponownie osiągnął najlepszy czas wykonania dzięki optymalizacji procesu wstawiania.

W przypadku algorytmów oznaczonych jako **sortB** i **sortC** zaobserwowano rozbieżności pomiędzy teoretyczną a zmierzoną liczbą porównań dla danych wejściowych posortowanych rosnąco. Teoretycznie, dla tego typu danych, liczba porównań powinna wynosić dokładnie $n - 1$, ponieważ każdy kolejny element powinien być porównany tylko raz ze swoim poprzednikiem, a nie przestawiany. Tymczasem, w wynikach pomiarów liczba porównań jest wyraźnie zawyżona, co sugeruje możliwy błąd w implementacji zmiennej **comps** odpowiedzialnej za zliczanie porównań w tych funkcjach. Taka nieprawidłowość może prowadzić nie tylko do fałszywego raportowania liczby operacji, ale również do rzeczywistego wydłużenia czasu sortowania.

6. Wnioski

Na podstawie poczynionych pomiarów można sprawdzić, czy poruszone wcześniej hipotezy okazały się poprawne:

- **H1 - złożoność algorytmów liniowego sortowania przez wstawianie** - Hipoteza ta zakładała, że algorytmy sortowania przez wstawianie liniowe (**sortA**, **sortB**, **sortC**, **sortD**) powinny wykazywać złożoność czasową rzędu $O(n^2)$ dla danych losowych i malejących oraz $O(n)$ dla danych posortowanych rosnąco. Wyniki eksperymentów potwierdziły tę zależność w większości przypadków - dla danych **losowych** oraz **posortowanych malejąco**, czasy sortowania rosły kwadratowo wraz ze wzrostem liczby elementów, co było również widoczne na wykresach czasów i liczby porównań. Dla danych **rosnących** algorytmy **sortA** i **sortD** faktycznie wykazały liniowy przyrost czasu oraz minimalną liczbę przestawień i porównań, co potwierdza oczekiwane zachowanie.
- **H2 - skuteczność sortowania binarnego** - Hipoteza ta odnosiła się do algorytmu sortowania przez wstawianie binarne (**sortE**), zakładając zmniejszenie liczby porównań (dzięki wyszukiwaniu pozycji wstawienia metodą binarną), przy jednoczesnym zachowaniu takiej samej liczby przestawień co w algorytmach liniowych. Liczba porównań w **sortE** była zauważalnie niższa niż w innych metodach, co zostało wyraźnie przedstawione na wykresach. Liczba przestawień pozostała niemal identyczna jak w **sortA** - **sortD**, co jest zgodne z teorią - mimo szybszego wyszukiwania pozycji, elementy nadal muszą być przesuwane ręcznie, co nie zmienia złożoności w zakresie operacji modyfikujących strukturę danych.
- **H3 - idealne zachowanie dla danych rosnących** - Hipoteza zakładała, że w przypadku danych posortowanych rosnąco, algorytmy powinny wykonywać dokładnie $n - 1$ porównań i 0 przestawień. Praktyczne wyniki częściowo potwierdziły tę hipotezę. W przypadku **sortA** i **sortD** liczba porównań i przestawień była zgodna z oczekiwaniami. Dla algorytmów **sortB** i **sortC** liczba porównań okazała się wyższa niż teoretyczne $n - 1$, a przestawienia nie były całkowicie wyeliminowane. Może to wskazywać na błąd w implementacji zmiennej **comps**, odpowiedzialnej za zliczanie porównań. Ta niezgodność podważa pełną realizację H3, ale jednocześnie wskazuje, że błąd nie wynika z koncepcji algorytmu, lecz z jego implementacji - co jest cenną informacją przy dalszym rozwoju lub poprawie kodu.

Analiza danych eksperymentalnych pozwoliła nie tylko na weryfikację postawionych hipotez, ale również na wyciągnięcie bardziej ogólnych wniosków dotyczących praktycznej efektywności algorytmów sortowania przez wstawianie. Przede wszystkim, mimo że wszystkie testowane algorytmy należą do tej samej klasy złożoności obliczeniowej $O(n^2)$ w przypadkach nieuporządkowanych danych), ich rzeczywista wydajność istotnie różni się w zależności od szczegółów implementacyjnych. Przykładowo, **sortA** i **sortD** - mimo podobnego schematu działania – wypadły nieco lepiej niż **sortB** i **sortC**, co może być związane z bardziej zoptymalizowanym zarządzaniem operacjami przesuwania i porównywania. Sugeruje to, że sposób zaimplementowania tych samych operacji ma realny wpływ na zużycie zasobów i czas wykonania, nawet jeśli algorytmy są teoretycznie równoważne. Sortowanie binarne (**sortE**) potwierdziło swoją przewagę w zakresie liczby porównań – zwłaszcza przy większych tablicach, gdzie różnica względem innych metod była najbardziej widoczna. Mimo to, ograniczenia tego podejścia wynikają z faktu, że rzeczywiste przesunięcia danych w tablicy nadal muszą być wykonywane liniowo, co nie pozwala znacząco poprawić ogólnej złożoności. Pojawiły się również pewne anomalie wynikające z błędów implementacyjnych - co unaocznia, jak duże znaczenie ma poprawna realizacja szczegółów algorytmu. Dotyczy to szczególnie przypadków, w których zliczanie operacji (porównań, przestawień) ma być częścią analizy - błędna logika zliczania może nie tylko fałszować dane, ale również wpływać na wydajność programu. Wyniki badań wskazują także na różne poziomy stabilności algorytmów względem rodzaju danych - algorytmy liniowe wykazują duży spadek wydajności przy danych posortowanych malejąco, podczas gdy algorytm binarny pozostaje bardziej odporny na takie zmiany, przynajmniej w kontekście liczby porównań. Jest to ważna obserwacja w kontekście doboru algorytmu do konkretnych zastosowań.

7. Podsumowanie

Celem projektu było przeanalizowanie efektywności różnych implementacji algorytmów sortowania przez wstawianie. Badania przeprowadzono w sposób systematyczny, testując zachowanie algorytmów dla różnych typów danych wejściowych oraz różnych rozmiarów tablic. Na podstawie wyników:

- Potwierdzono teoretyczne złożoności algorytmów w większości przypadków.
- Wykazano, że poprawność implementacji ma kluczowe znaczenie dla wiarygodności wyników - zarówno w kontekście czasu wykonania, jak i liczby operacji.
- Udowodniono, że wyszukiwanie pozycji wstawienia za pomocą metody binarnej rzeczywiście zmniejsza liczbę porównań, jednak nie zmienia złożoności całkowitej, gdyż przesuwanie elementów nadal odbywa się liniowo.
- Zidentyfikowano nieprawidłowości w implementacjach **sortB** i **sortC**, które wpłynęły negatywnie na wydajność i dokładność pomiarów w przypadku danych posortowanych rosnąco.
- Pokazano, że nawet w obrębie tej samej klasy algorytmicznej można osiągnąć wyraźne różnice wydajnościowe w zależności od detali implementacyjnych.

Wnioski z projektu podkreślają wagę precyzyjnego kodowania, testowania i interpretacji wyników w badaniach nad efektywnością algorytmów. Uzyskane rezultaty mogą być podstawą do dalszej analizy i optymalizacji algorytmów sortujących, a także pokazują, że pozornie prosty algorytm może ujawniać złożone zachowania w zależności od charakterystyki danych wejściowych.

8. Literatura

miroslawzelent.pl/kurs-c++/sortowanie-zlozonosc-algorytmow/

https://en.wikipedia.org/wiki/Insertion_sort

https://en.wikipedia.org/wiki/Sorting_algorithm

https://en.wikipedia.org/wiki/Comparison_sort